Correct. The ToR should explicitly identify **which existing project/app each architectural component is reverse-engineered or benchmarked from**, rather than mentioning the references only generally.

I would add a dedicated **“Reverse-Engineering Source & Design Lineage”** section and also identify the source in each relevant module.

Recommended replacement section

Mega3 Reverse-Engineering Source & Design Lineage

REVERSE-ENGINEERING SOURCE & DESIGN LINEAGE

Mega3 is an original implementation that studies and adapts publicly observable architectural concepts, engineering patterns, and documented behaviors from established MEV, EVM, trading, and telemetry systems.

The development agent SHALL preserve explicit attribution of the architectural inspiration for each subsystem.

Mega3 must **not copy proprietary source code, private algorithms, credentials, or confidential implementation details**. Reverse engineering in this ToR means architectural study, benchmarking, interface analysis, and independent reimplementation of publicly observable concepts.

---

1. Artemis — Paradigm

Reference Project/App: Paradigm **Artemis**

Mega3 Components Influenced:

- `mega3-ingest`
- collector architecture
- event-driven ingestion
- typed internal events
- separation of data collection from strategy execution
- modular protocol adapters

Artemis → Mega3 adaptation

```
Artemis
   ↓
Protocol/Data Collectors
   ↓
Typed Events
   ↓
Independent Processing Modules

Mega3
   ↓
CEX Collector + BSC Collector
   ↓
Typed Mega3 Events
   ↓
Solver / Simulation / Execution
```

Design principle adopted:

> Network ingestion must remain decoupled from computationally intensive strategy and execution processes.

Mega3 SHALL independently implement this architecture using its own Rust modules and interfaces.

---

2. Rusty-Sando

Reference Project/App: Rusty-Sando

Mega3 Components Influenced:

- `mega3-simulation`
- EVM state management
- `revm`-based simulation
- cached state architecture
- rapid transaction playback
- simulation isolation

Rusty-Sando → Mega3 adaptation

```
Rusty-Sando
      ↓
Cached EVM State
```

```
      ↓
Fast Simulation Context
      ↓
Transaction Playback

Mega3
      ↓
Persistent Base CacheDB
      ↓
Ephemeral Simulation Context
      ↓
Opportunity Simulation
```

The objective is to minimize unnecessary state reconstruction and memory overhead.

Important: the `<120 µs` simulation target is a Mega3 engineering target derived from the intended high-performance architecture. It must be independently benchmarked and must not be represented as an achieved Rusty-Sando performance figure unless experimentally demonstrated.

---

3. Flashbots MEV-Share

Reference Project/App: Flashbots MEV-Share

Mega3 Components Influenced:

- `mega3-executor`
- private transaction architecture
- bundle concepts
- opportunity targeting
- execution privacy
- builder/relay abstraction
- inclusion monitoring

Flashbots → Mega3 adaptation

```
Flashbots MEV-Share
        ↓
Private MEV Orderflow
        ↓
Bundle / Transaction Submission
```

```
        ↓
Builder
        ↓
Block Inclusion

Mega3
        ↓
Private Execution Request
        ↓
BuilderRelay abstraction
        ↓
Multiple BSC-compatible Relays
        ↓
Block Inclusion
```

Mega3 SHALL implement relay adapters independently.

No assumption should be made that an Ethereum Flashbots interface is directly compatible with BSC.

Each BSC relay/builder integration must be separately verified.

---

4. Paradigm / Artemis Telemetry Philosophy

Reference Project/App: Paradigm Artemis

Mega3 Component:

`mega3-dashboard/`

The dashboard architecture takes inspiration from Artemis' emphasis on modular observability and high-frequency system information.

Mega3 extends this concept through:

- asynchronous telemetry
- binary transport
- WebSocket streaming
- batched updates
- in-memory frontend state
- execution opportunity matrices

The dashboard remains an **observability subsystem**, not part of the transaction hot path.

---

5. revm Ecosystem

Reference Technology: revm

Mega3 Component:

`mega3-simulation/`

revm provides the EVM execution foundation.

Mega3 SHALL use established EVM execution libraries rather than creating a proprietary EVM implementation.

The agent must pin and document the exact compatible version used by Mega3.

---

6. Alloy Ecosystem

Reference Technology: Alloy

Mega3 Components:

- BSC RPC interaction
- Ethereum/BSC primitives
- transaction structures
- provider interfaces
- ABI-related infrastructure

Alloy is a dependency foundation rather than a reverse-engineered application.

Mega3 SHALL use standard, maintained cryptographic and blockchain primitives instead of implementing custom cryptographic mechanisms.

---

7. Binance Market Data

Reference Application/API: Binance market-data infrastructure

Mega3 Component:

```
mega3-ingest/cex_stream.rs
```

Mega3 uses publicly available CEX market-data streams as an external predictive signal source.

The CEX stream is transformed into:

```
External JSON/WebSocket
        ↓
Decoder
        ↓
Validation
        ↓
Typed Mega3 Event
        ↓
Divergence Engine
```

CEX ingestion must remain independent of the execution engine.

---

8. PancakeSwap V2 / V3

Reference Protocols: PancakeSwap V2 and PancakeSwap V3

Mega3 Components:

```
mega3-simulation/
mega3-ingest/
mega3-solver/
```

These protocols provide the primary BSC liquidity venues for the initial Mega3 arbitrage model.

Mega3 shall independently implement:

- reserve-based V2 pricing
- concentrated-liquidity V3 pricing
- decimal normalization
- fee-tier handling
- token ordering
- slippage modeling
- pool-state simulation

Pool addresses and parameters must be independently verified against current BSC deployment data.

9. Pimlico / ERC-4337

Reference Platform: Pimlico

Reference Technology: ERC-4337 Account Abstraction / Paymaster

Mega3 Relationship:

Pimlico is **not the model for the MEV hot execution path**.

Instead, it is treated as an optional infrastructure reference for:

- account abstraction
- sponsored user transactions
- wallet onboarding
- non-HFT application operations

Mega3's high-frequency execution path remains independently designed around its selected execution-wallet and private transaction model.

Therefore:

```
Pimlico / ERC-4337
        │
        ├── User transactions
        ├── Sponsored operations
        └── Administrative operations

NOT
        │
        ▼
High-frequency MEV hot path
```

Any future decision to place account abstraction into the execution path must be supported by measured latency and reliability evidence.

---

10. Reverse-Engineering Attribution Matrix

Mega3 Component	Reference Project / App / Technology	Primary Design Influence

`mega3-core`	Mega3 original architecture	Unified internal type system
`mega3-ingest`	**Paradigm Artemis**	Async collectors / event architecture
`cex_stream.rs`	**Binance market-data infrastructure**	High-frequency market stream
`bsc_stream.rs`	BSC/RPC ecosystem	On-chain event ingestion
`recorder.rs`	**Artemis-style event architecture + Mega3 original design**	Divergence recording
`mega3-simulation`	**Rusty-Sando + revm**	Cached EVM simulation
`db_lifecycle.rs`	**Rusty-Sando**	State lifecycle optimization
`math_adapter.rs`	**PancakeSwap V2/V3**	AMM pricing mathematics
`mega3-solver`	**Artemis / Rusty-Sando concepts + Mega3 original optimization**	Opportunity/path optimization
`mega3-executor`	**Flashbots MEV-Share**	Private MEV submission architecture
`builder_relay.rs`	**Flashbots-style relay abstraction**	Multi-relay execution
`mega3-research/scanner.rs`	Mega3 original	Historical competitor/tip analysis
`mega3-dashboard`	**Paradigm Artemis telemetry philosophy**	High-frequency observability
Binary BSON telemetry	Mega3 implementation choice	Compact asynchronous telemetry
Account abstraction	**Pimlico / ERC-4337**	Optional non-hot-path sponsorship

EVM execution	**revm**	Production EVM simulation foundation
Blockchain primitives	**Alloy**	BSC/EVM data and transaction primitives

---

11. Required Attribution Rule for the Development Agent

Every major Mega3 module SHALL contain a short source-lineage comment in its documentation.

For example:

```
//! Mega3 Ingestion Layer
//!
//! Architectural lineage:
//! - Paradigm Artemis: modular asynchronous collector architecture.
//! - Mega3 original: BSC/CEX dual-vector event model.
//!
//! This module is an independent implementation and does not copy
//! proprietary source code.
```

For the simulator:

```
//! Mega3 Simulation Layer
//!
//! Architectural lineage:
//! - Rusty-Sando: cached EVM-state / rapid simulation concepts.
//! - revm: EVM execution engine.
//! - Mega3 original: persistent BSC opportunity simulation model.
```

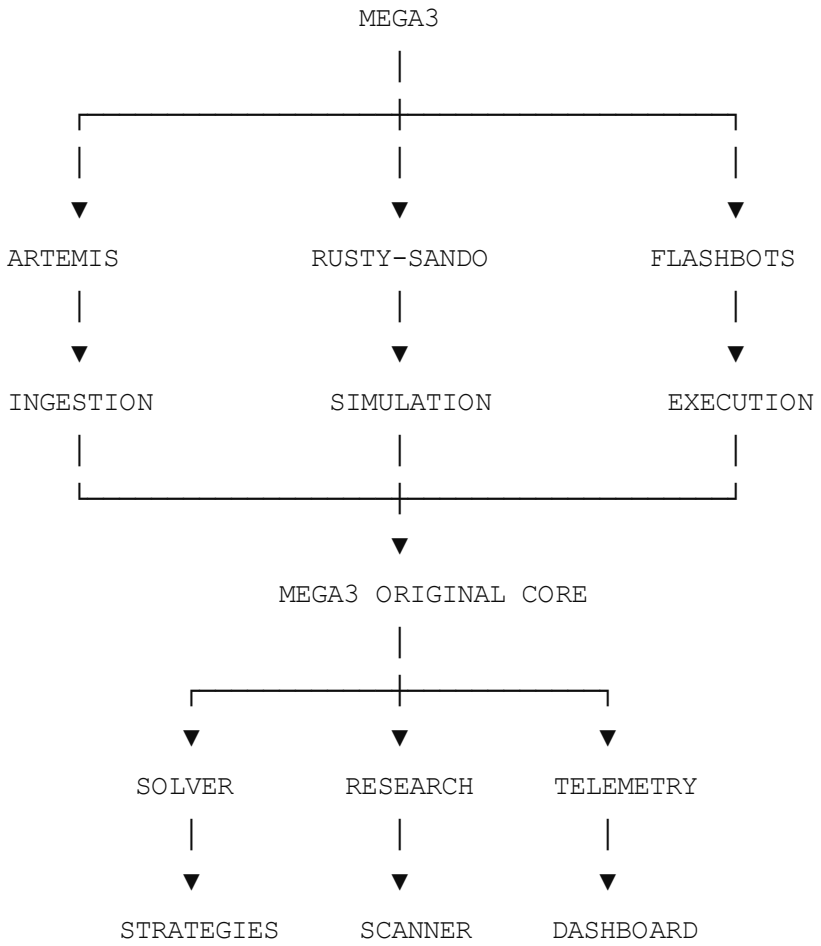
For the executor:

```
//! Mega3 Execution Layer
//!
//! Architectural lineage:
//! - Flashbots MEV-Share: private MEV transaction/bundle concepts.
//! - Mega3 original: BSC multi-relay execution abstraction.
```

---

12. Final Architectural Lineage

The overall Mega3 architecture can therefore be represented as:



Architectural rule

The referenced projects are **design lineage and engineering benchmarks**, not code sources.

Mega3 SHALL independently implement its own:

- interfaces
- data structures
- algorithms
- configuration
- strategy logic
- execution logic
- telemetry system
- testing framework

The purpose of reverse engineering is to identify proven architectural patterns, understand their strengths and limitations, and produce a Mega3-specific production implementation.

I would insert this section into the rewritten ToR immediately after the Executive Mandate and also keep the attribution matrix near the end. That makes it unmistakable to the development agent that **Artemis → ingestion/telemetry, Rusty-Sando → simulation, Flashbots MEV-Share → private execution, Binance → CEX data, PancakeSwap → AMM mathematics, revm/Alloy → infrastructure, and Pimlico/ERC-4337 → optional non-hot-path AA**.